

Introduction to Optimization in ML

Nirupam Gupta

Department of Computer Science

UNIVERSITY OF COPENHAGEN



Mathematical optimization

$$\begin{array}{ll}\text{Minimize} & f(w) \\ \text{Subject to} & f_i(w) \leq 0 \quad i = 1, \dots, p \\ & g_i(w) = 0 \quad i = 1, \dots, q\end{array}$$

- $w \in \mathbb{R}^d$ represents scalar **variables** $w_1, \dots, w_d \in \mathbb{R}$, to be computed.
- $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is the **objective function**, to be minimized.
- $\{f_1, \dots, f_p\}$ with $f_i : \mathbb{R}^d \rightarrow \mathbb{R}$ are the **inequality constraints**.
- $\{g_1, \dots, g_q\}$ with $g_i : \mathbb{R}^d \rightarrow \mathbb{R}$ are the **equality constraints**.

Minimizing $f(x)$ is equivalent to maximizing $-f(x)$.

Linear classification

Solving **empirical risk minimization** (ERM) for designing classifiers.

Example: **Linear Classifier**

- n **samples** (input-output pairs) $(x_1, y_1), \dots, (x_n, y_n) \in \mathbb{R}^m \times \{-1, 1\}$.
- Linear **hypothesis class** $\mathcal{H} = (h : x \mapsto \text{sign}(w^\top x + w_o) \mid (w, w_o) \in \mathbb{R}^m \times \mathbb{R})$.
- For each sample (x_i, y_i) , **define loss** by $f_i(w, w_o) := \mathbb{1}[h(x_i) \neq y_i]$ where $\mathbb{1}[\cdot]$ is the indicator function

Solve the following **optimization problem**:

$$\begin{array}{ll} \text{Minimize} & f(w, w_o) := \frac{1}{n} \sum_{i=1}^n f_i(w, w_o) \\ \text{Subject to} & \|(w, w_o)\|_0 - k \leq 0 \end{array} \quad (\text{ERM})$$

The inequality constraint ensures sparsity in the weights when $k < m$.

General optimization framework in ML

Training **artificial neural networks**

- n **samples** (input-output pairs) $(x_1, y_1), \dots, (x_n, y_n) \in \mathbb{R}^m \times \{-1, +1\}$.
- Hypothesis $h(x) : \mathbb{R}^m \rightarrow \mathbb{R}^d$ is typically **non-linear**, determined by tunable parameters $w \in \mathbb{R}^d$.
- For each sample (x_i, y_i) , define loss by **cross-entropy** function

$$f_i(w) := \mathbb{1}[y = +1] \frac{e^{[h(x)]_1}}{e^{[h(x)]_1} + e^{[h(x)]_2}} + \mathbb{1}[y = -1] \frac{e^{[h(x)]_2}}{e^{[h(x)]_1} + e^{[h(x)]_2}}$$

where $[\cdot]_k$ denotes the k -th coordinate value of the vector.

Solve the following **optimization problem**:

$$\begin{array}{ll} \text{Minimize} & f(w) := \frac{1}{n} \sum_{i=1}^n f_i(w) \\ \text{Subject to} & \|w\|_0 - k \leq 0 \end{array} \quad (\text{ERM})$$

Inequality constraint ensures sparsity of the network when $k < m$.

Solving an optimization problem

$$\begin{array}{ll}\text{Minimize} & f(w) \\ \text{Subject to} & \underset{w \in \mathbb{R}^d}{f_i(w)} \leq 0 \quad i = 1, \dots, p \\ & g_i(w) = 0 \quad i = 1, \dots, q\end{array}$$

In general, it is *practically impossible* to solve the above problem.

However, many optimization problems can be solved efficiently, e.g., **convex optimization** problems.

- **Objective function** $f(w)$ and **inequality constraints** $f_1(w), \dots, f_p(w)$ are **convex** functions -

$$f(\theta w_1 + (1 - \theta)w_2) \leq \theta f(w_1) + (1 - \theta)f(w_2), \quad \text{for all } w_1, w_2 \in \mathbb{R}^d \text{ and } \theta \in [0, 1].$$

- **Equality constraints** $g_1(w), \dots, g_q(w)$ are **affine** functions:

$$g_i(w) := a_i^\top w + b_i, \quad \text{where } (a_i, b_i) \in \mathbb{R}^d \times \mathbb{R}.$$

Quadratic programming (QP)

Convex optimization with quadratic objective and affine constraint functions.

$$\begin{array}{ll} \underset{w \in \mathbb{R}^d}{\text{Minimize}} & f(w) := w^\top P w + Q w \\ \text{Subject to} & G w \preceq c \\ & H w = d \end{array} \quad ; \quad \begin{array}{l} G \in \mathbb{R}^{p \times d}, \quad c \in \mathbb{R}^p \\ H \in \mathbb{R}^{q \times d}, \quad d \in \mathbb{R}^q \end{array}$$

where $P \in \mathbf{S}_+^d$ (i.e., symmetric positive semidefinite) and $Q \in \mathbb{R}^{1 \times d}$.

Example: Support vector machine (SVM).

Lagrangian function

Primal optimization problem:

$$\begin{array}{ll} \underset{w \in \mathbb{R}^d}{\text{Minimize}} & f(w) \\ \text{Subject to} & f_i(w) \leq 0 \quad i = 1, \dots, p \\ & g_i(w) = 0 \quad i = 1, \dots, q \end{array} \quad (\text{Primal})$$

We define the **Lagrangian** $\mathcal{L} : \mathbb{R}^d \times \mathbb{R}^p \times \mathbb{R}^q \rightarrow \mathbb{R}$ of the above problem to be

$$\mathcal{L}(w, \lambda, \nu) = f(w) + \sum_{i=1}^p \lambda_i f_i(w) + \sum_{i=1}^q \nu_i g_i(w).$$

$\lambda = (\lambda_1, \dots, \lambda_p)$ and $\nu = (\nu_1, \dots, \nu_q)$ are called **dual variables** or **Lagrange multipliers**.

Lagrange dual function

The **Lagrange dual function** $\phi : \mathbb{R}^p \times \mathbb{R}^q \rightarrow \mathbb{R}$ is defined to be

$$\phi(\lambda, \nu) := \min_{w \in \mathbb{R}^d} \mathcal{L}(w, \lambda, \nu) = \min_{w \in \mathbb{R}^d} \left(f(w) + \sum_{i=1}^p \lambda_i f_i(w) + \sum_{i=1}^q \nu_i g_i(w) \right).$$

$\phi(\lambda, \nu)$ is a concave function.

Let p^* be the optimal value of the primal problem (i.e., $p^* = \min f(w)$ subject to the constraints). For all $\lambda \succeq 0$ and ν ,

$$p^* \geq \phi(\lambda, \nu).$$

Lagrange dual optimization problem

The **Lagrange dual problem** is defined to be

$$\begin{array}{ll} \text{Maximize} & g(\lambda, \nu) \\ \text{Subject to} & \lambda \succeq 0 \end{array} \quad (\text{Dual})$$

Suppose that d^* is the optimal value of (Dual) (i.e., $d^* = \max g(\lambda, \nu)$ s.t. $\lambda \succeq 0$.) Then,

$$p^* \geq d^*.$$

This property is called **weak duality**. It holds true even if the primal problem is nonconvex.

Strong duality

We have **strong duality** when

$$p^* = d^*.$$

Strong duality usually (but not always) holds when the **primal problem is convex**.

Conditions, beyond convexity, for strong duality are called **constraint qualifications**.

Slater's condition: There exists $w \in \mathbb{R}^d$ such that

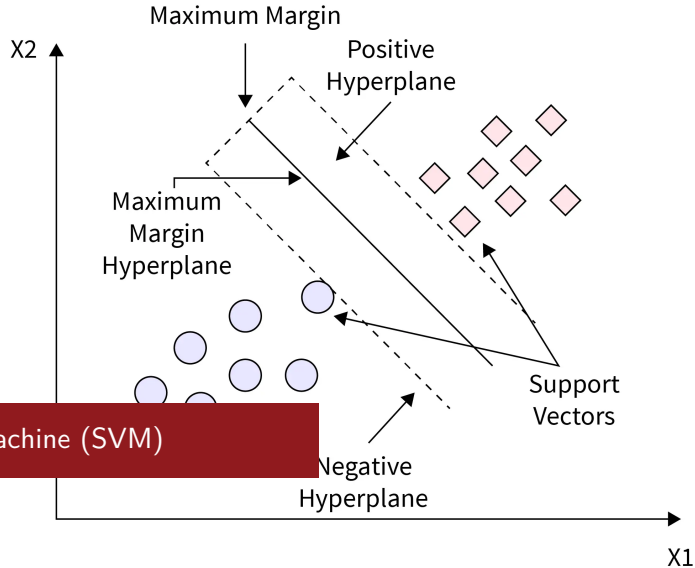
$$\begin{aligned} f_i(w) &< 0, & i = 1, \dots, p \\ a_i^\top w + b_i &< 0, & i = 1, \dots, q \end{aligned}$$

KKT optimality conditions

KKT necessary conditions for optimality (under differentiability and strong duality):

$$\begin{aligned} f_i(w^*) &\leq 0, & i = 1, \dots, p \\ g_i(w^*) &= 0, & i = 1, \dots, q \\ \lambda^* &\succeq \mathbf{0} \\ \lambda_i^* f_i(w^*) &= 0, & i = 1, \dots, p \\ \nabla_w f(w^*) + \sum_{i=1}^p \lambda_i^* \nabla_w f_i(w^*) + \sum_{i=1}^q \nu_i^* \nabla_w g_i(w^*) &= \mathbf{0} \end{aligned} \tag{KKT}$$

If the **primal** is **convex** then **KKT conditions are sufficient** for optimality.



Support Vector Machine (SVM)

Linear SVM as QP

$$\begin{array}{ll}\text{Maximize} & \frac{\rho}{\|w\|} \\ \text{Subject to} & y_i(w^\top x_i + b) \geq \rho \quad , \quad i = 1, \dots, n \\ & \rho > 0\end{array}$$

is reducible to

$$\begin{array}{ll}\text{Maximize} & \frac{1}{\|w\|} \\ \text{Subject to} & y_i(w^\top x_i + b) \geq 1 \quad , \quad i = 1, \dots, n\end{array}$$

This can be solved using the following **quadratic programming** (QP):

$$\begin{array}{ll}\text{Minimize} & \frac{1}{2} \|w\|^2 \\ \text{Subject to} & 1 - y_i(w^\top x_i + b) \leq 0 \quad , \quad i = 1, \dots, n\end{array} \quad \text{(Linear SVM)}$$

Lagrange dual of linear SVM

Lagrange dual function:

$$\mathcal{L}(w, b, \lambda) := \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \lambda_i (1 - y_i(w^\top x_i + b)).$$

Dual optimization problem:

$$\begin{array}{ll} \underset{\lambda \in \mathbb{R}^n}{\text{Maximize}} & \phi(\lambda) := \sum_{i=1}^n \lambda_i - \frac{1}{2} \left\| \sum_{i=1}^n \lambda_i y_i x_i \right\|^2 \\ \text{Subject to} & \lambda \succeq \mathbf{0} \\ & \sum_{i=1}^n \lambda_i y_i = 0 \end{array} \quad \text{(Dual of Linear SVM)}$$

Let (w^*, b^*) and λ^* be solutions to the (Linear SVM) and (Dual of Linear SVM), respectively.

KKT optimality conditions of linear SVM

$$\begin{aligned}
 1 - y_i(\langle w^*, x_i \rangle + b^*) &\leq 0, & i = 1, \dots, n \\
 \lambda^* &\succeq \mathbf{0} \\
 \lambda_i^* (1 - y_i(\langle w^*, x_i \rangle + b^*)) &= 0, & i = 1, \dots, n \\
 w^* - \sum_{i=1}^n \lambda_i^* y_i x_i &= \mathbf{0} \\
 \sum_{i=1}^n \lambda_i^* y_i &= 0
 \end{aligned}
 \tag{KKT Linear SVM}$$

Support vectors: set of points (x_i, y_i) for which $\lambda_i^* > 0$. Due to complementary slackness, $y_i(\langle w^*, x_i \rangle + b) = 1$ (or $\langle w^*, x_i \rangle + b = y_i$) for all support vectors.

$w^* = \sum_{i \in SV} \lambda_i^* y_i x_i$, where $SV \subseteq [n]$ denotes the index set of all the support vectors.

For any $i \in SV$, $\langle w^*, x_i \rangle + b = y_i$. Thus, $b = y_i - \langle w^*, x_i \rangle = y_i - \sum_{j=1}^n \lambda_j^* y_j \langle x_j, x_i \rangle$.

The largest margin of separation is equal to $\frac{1}{\sum_{i \in SV} \lambda_i^*}$.

Nonlinearly separable points

Suppose that the dataset $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ is NOT linearly separable. That is, the convex hulls of $S^+ = \{x \mid (x, y) \in S, y = +1\}$ and $S^- = \{x \mid (x, y) \in S, y = -1\}$ are NOT disjoint.

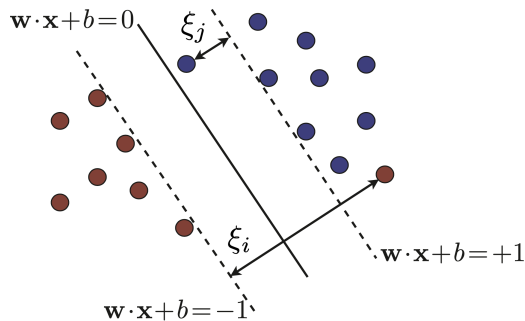


Figure: Nonlinearly separable points.

Linear SVM with soft margin

Slack variables. Introduce nonnegative variables $\xi_i, i = 1, \dots, n$ such that for all i ,

$$y_i(w^T x_i + b) + \xi_i \geq 1$$

We call ξ_i 's as *slack variables*.

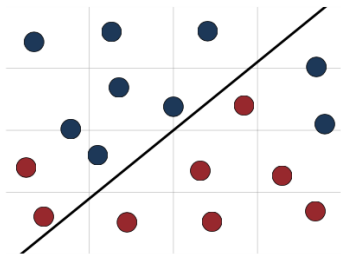
In this case, we train a linear SVM with **soft margin** using the following QP:

$$\begin{array}{ll} \underset{\xi \in \mathbb{R}^n, w \in \mathbb{R}^m, b \in \mathbb{R}}{\text{Minimize}} & \frac{1}{2} \|w\|^2 + \Psi(\xi_1, \dots, \xi_n) \\ \text{Subject to} & 1 - y_i(w^T x_i + b) - \xi_i \leq 0 \quad , \quad i = 1, \dots, n \\ & -\xi \preceq \mathbf{0} \end{array} \quad (\text{Soft-margin SVM})$$

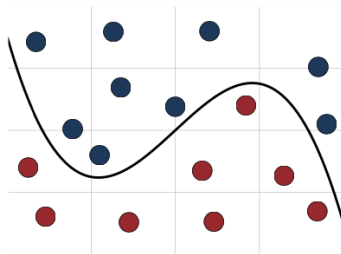
where, $\Psi(\xi_1, \dots, \xi_n)$ is convex and typically taken to be $c \sum_{i=1}^n \xi_i^r$ for $r \geq 1$.

We want ξ to be sparse. This can be approximately obtained by using $r = 1$.

Linear classifiers can be suboptimal



(a)



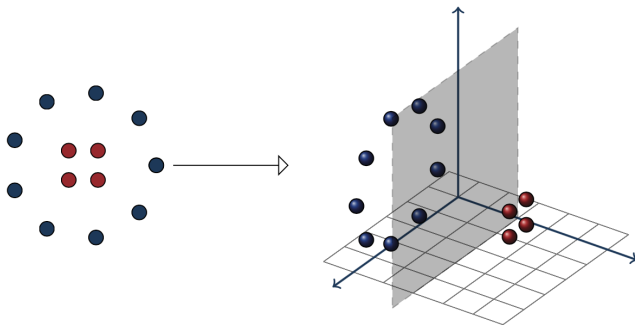
(b)

Figure: (a) Linear classifier. (b) Nonlinear classifier.

Nonlinear transformation for inducing linear separability

Nonlinear mapping to a higher dimensional space.

For example, in the case below with input space $\mathcal{X} \subset \mathbb{R}^2$, by mapping x to $\Psi(x) = ([x]_1, [x]_2, \|x\|)$ we obtain linear separability.



Linear SVM with input space transformation

Consider **feature mapping** $\Psi : \mathcal{X} \rightarrow \mathcal{Z}$, where $\mathcal{Z}$ is referred to as the **feature space**.

Dual problem of **linear SVM over transformed data points**:

$$\begin{array}{ll} \underset{\lambda \in \mathbb{R}^n}{\text{Maximize}} & \phi(\lambda) := \sum_{i=1}^n \lambda_i - \frac{1}{2} \left\| \sum_{i=1}^n \lambda_i y_i \Psi(x_i) \right\|^2 \\ \text{Subject to} & 0 \leq \lambda_i \leq c \text{ and } \sum_{i=1}^n \lambda_i y_i = 0 \end{array}$$

By analogy to original SVM problem, $w^* = \sum_{i=1}^n \lambda_i^* y_i \Psi(x_i) \in \mathcal{Z}$. For i such that $0 < \lambda_i^* < c$ we obtain $b^* = y_i - \sum_{j=1}^n \lambda_j^* y_j \langle \Psi(x_i), \Psi(x_j) \rangle$.

Hypothesis: $h(x) = \text{Sign}(\langle w^*, \Psi(x) \rangle + b^*)$.

Caveat: Computational cost for $\Psi(x)$ is in $\mathcal{O}(\dim(\mathcal{Z}))$ and can be prohibitively high in practice.

Kernels: Efficient incorporation of nonlinear transformation

We can write $\phi(\lambda) := \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i,j=1}^n \lambda_i \lambda_j y_i y_j \langle \Psi(x_i), \Psi(x_j) \rangle$.

Moreover, the hypothesis $h(x) = \text{Sign} \left(\sum_{i=1}^n \lambda_i^* y_i \langle \Psi(x_i), \Psi(x) \rangle + b^* \right)$

These computations involving inner-products can be performed without explicitly computing Ψ .

Kernels: A function $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$.

K is **positive definite symmetric** (PDS) if for any $\{x_1, \dots, x_n\} \subset \mathcal{X}$, the *Gram matrix* $\mathbf{K} = [K(x_i, x_j)]_{ij}$ is (symmetric) positive semi-definite.

Theorem. If K is **PDS** then K defines an inner product in a Hilbert space $\mathcal{Z}$, and

there exists $\Psi : \mathcal{X} \rightarrow \mathcal{Z}$ such that $K(x, x') = \langle \Psi(x), \Psi(x') \rangle_{\mathcal{Z}}$.

Linear SVM with kernel K

Replacing $\langle \Psi(x_i), \Psi(x_j) \rangle$ by $K(x_i, x_j)$ in the dual SVM problem we obtain:

$$\begin{array}{ll} \underset{\lambda \in \mathbb{R}^n}{\text{Maximize}} & \phi(\lambda) := \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i,j=1}^n \lambda_i \lambda_j y_i y_j K(x_i, x_j) \\ \text{Subject to} & 0 \leq \lambda_i \leq c \\ & i=1, \dots, n \\ & \sum_{i=1}^n \lambda_i y_i = 0 \end{array}$$

The resulting hypothesis is given by

$$h(x) = \text{Sign} \left(\sum_{i=1}^n \lambda_i^* y_i K(x_i, x) + b^* \right),$$

where $b^* = y_i - \sum_{j=1}^n \lambda_j^* y_j K(x_i, x_j)$ with $i \in [n]$ such that $0 < \lambda_i^* < c$.

Examples of PDS kernels

- **Polynomial.** For $a \in \mathbb{R}$, polynomial kernel of degree $k \geq 1$ is $K(x, x') = (x^\top x' + a)^k$.
- **Exponential.** For $a \in \mathbb{R}$, exponential kernel is $K(x, x') = \exp\left(\frac{x^\top x'}{a^2}\right)$.
- **Normalized.** For a PDS K , its normalized kernel $\hat{K}$ (defined below) is also PDS.

$$\hat{K}(x, x') = \begin{cases} 0 & , \quad K(x, x) = 0 \vee K(x', x') = 0 \\ \frac{K(x, x')}{\sqrt{K(x, x) K(x', x')}} & , \quad \text{o.w.} \end{cases}$$

- **Gaussian.** For $a \in \mathbb{R}$, Gaussian kernel (or *radial basis function* (RBF)) is

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2a^2}\right).$$

- **Sigmoid.** For $a, b \in \mathbb{R}_+$, a sigmoidal kernel is $K(x, x') = \tanh(a x^\top x' + b)$.

Reproducing property of PDS kernels

Kernels can be used to define a large class of functions on $\mathcal{X}$.

If $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is PDS, then

- For all $x \in \mathcal{X}$, $K(x, \cdot) \in \mathcal{Z}$.
- $\mathcal{Z}$ is a **reproducing kernel Hilbert space** (RKHS) associated to K , meaning that any $z \in \mathcal{Z}$ defines a mapping from $\mathcal{X}$ to $\mathbb{R}$ whose value at any $x \in \mathcal{X}$ is given by a linear combination:

$$z(x) = \langle z, K(x, \cdot) \rangle_{\mathcal{Z}}.$$

For a PDS K , we can define a feature mapping: $\Psi(x) = K(x, \cdot)$.

For SVM with K , an optimal hyperplane (ignoring the offset) is given by $z^* := \sum_{i=1}^n \lambda_i y_i K(x_i, \cdot)$.

Beyond SVM: wider application of kernels

We can also apply kernels in other machine learning tasks like regression, dimensionality reduction or clustering:

Consider the following optimization problem associated with a mapping z induced over $\mathcal{X}$ by a **PDS kernel** $K : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$.

$$\underset{z \in \mathcal{Z}}{\text{Minimize}} \quad M(\|z\|) + \mathcal{L}(z(x_1), \dots, z(x_n)), \quad (\text{Opt K})$$

where $M : \mathbb{R} \rightarrow \mathbb{R}$ is monotonically non-decreasing and $\mathcal{L} : \mathbb{R}^n \rightarrow \mathbb{R}$.

Representer theorem: (Opt K) admits a solution $z^* = \sum_{i=1}^n \alpha_i K(x_i, \cdot)$.

Limitations of kernel trick

- **Kernel selection is challenging.** Requires domain expertise and lot of experimentation.
- **Not very scalable.** Can be expensive to implement on large datasets.
Can be tackled to certain extent through *approximate kernel feature maps*.
- **High sensitivity to kernel parameters.** A small change in kernel parameters can drastically change SVM's performance.

Logistic regression

Consider a dataset $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ such that $x_i \in \mathbb{R}^m$ and $y_i \in \{0, 1\}$ for all i .

For $w \in \mathbb{R}^m$ and $b \in \mathbb{R}$, **define** $z_i = w^\top x_i + b$ and $p_i = \text{Sigmoid}(z_i) = \frac{1}{1 + \exp(-z_i)}$.

p_i is the probability that the true label is 1 for x_i for a model parameterized by $w \in \mathbb{R}^m$ and $b \in \mathbb{R}$.

Cross-entropy loss. For each sample (x_i, y_i) , define

$$\ell_i(w, b) = - \left[y_i \log \left(\frac{1}{1 + \exp(-z_i)} \right) + (1 - y_i) \log \left(\frac{\exp(-z_i)}{1 + \exp(-z_i)} \right) \right]$$

Optimization problem.

$$\underset{w \in \mathbb{R}^m, b \in \mathbb{R}}{\text{Minimize}} \quad \mathcal{L}(w, b) := \frac{1}{n} \sum_{i=1}^n \ell_i(w, b)$$

The loss function $\mathcal{L}(w, b)$ is differentiable and convex.

Unconstrained optimization methods

Optimization problem:

$$\underset{w \in \mathbb{R}^d}{\text{Minimize}} \quad f(w)$$

Descent methods. Initial guess w_1 , updated iteratively as $w_{t+1} = w_t + u_t$ to generate a *relaxation sequence* $\{f(w_t)\}_{t=1}^{\infty}$, i.e., $f(w_{t+1}) \leq f(w_t)$.

If $f(w)$ is lower bounded for all $w \in \mathbb{R}^d$, the above sequence converges.

Gradient descent method

Method of gradient descent. For $\gamma_t \in \mathbb{R}_{++}$, update rule: $w_{t+1} = w_t - \gamma_t \nabla f(w_t)$.

$$f(w_{t+1}) = f(w_t) - \gamma_t \|\nabla f(w_t)\|^2 + o(\gamma_t \|\nabla f(w_t)\|).$$

For $\gamma_t = \gamma < \frac{1}{L}$, we have the following convergence guarantees:

For any Lipschitz smooth f : $\frac{1}{T} \sum_{t=1}^T \|\nabla f(w_t)\| \in \mathcal{O}\left(\frac{1}{\sqrt{T}}\right)$.

Lipschitz smooth and convex f : $f(w_T) - f^* \in \mathcal{O}\left(\frac{1}{T}\right)$.

Lipschitz smooth and μ -strongly convex f : $f(w_T) - f^* \in \mathcal{O}\left(\exp\left(-\frac{1}{\kappa} T\right)\right)$, where $\kappa = \frac{L}{\mu}$ is the *condition number* of $f(w)$.

Method of stochastic gradient descent (SGD)

In ML the loss function is the sum of point-wise loss functions: $\mathcal{L}(w) := \frac{1}{n} \sum_{i=1}^n \ell_i(w)$.

Gradient descent does not scale well with n . A more practical approach:

$$g_t = \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(w_t),$$

where B is a random subset of S called a **batch**. Given w_t , we have: $\mathbb{E}[g_t] = \nabla \mathcal{L}(w_t)$ and we assume: $\mathbb{E}_{i \sim \mathcal{U}[n]} \left[\|\nabla \ell_i(w_t) - \nabla \mathcal{L}(w_t)\|^2 \right] \leq \sigma^2$.

SGD update rule: $w_{t+1} = w_t - \gamma_t g_t$.

For $\gamma_t = \frac{1}{t}$, with $\mathcal{L}$ being Lipschitz smooth and strongly convex, we obtain that

$$\mathbb{E}[\mathcal{L}(w_{T+1}) - \mathcal{L}^*] \in \mathcal{O} \left(\kappa \frac{\sigma^2}{|B|} \cdot \frac{\log T}{T} \right).$$

Logistic regression with l_2 -regularization

Consider a dataset $S = \{(x_1, y_1), \dots, (x_n, y_n)\}$ such that $x_i \in \mathbb{R}^m$ and $y_i \in \{0, 1\}$ for all i .

For $w \in \mathbb{R}^m$ and $b \in \mathbb{R}$, **define** $z_i = w^\top x_i + b$ and $p_i = \text{Sigmoid}(z_i) = \frac{1}{1 + \exp(-z_i)}$.

Cross-entropy loss. For each sample (x_i, y_i) , define

$$\ell_i(w, b) = - \left[y_i \log \left(\frac{1}{1 + \exp(-z_i)} \right) + (1 - y_i) \log \left(\frac{\exp(-z_i)}{1 + \exp(-z_i)} \right) \right]$$

Regularized ERM:

$$\underset{w \in \mathbb{R}^m, b \in \mathbb{R}}{\text{Minimize}} \quad \mathcal{L}(w, b) := \frac{1}{n} \sum_{i=1}^n \ell_i(w, b) + \frac{\mu}{2} (\|w\|^2 + b^2)$$

The loss function $\mathcal{L}(w, b)$ is Lipschitz smooth and strongly convex.